

Problem Set 5: Trees, Forests, and Fairness in ML

Your mission for this problem set is to use your knowledge of tree-based methods and supervised learning to -- among other things! -- explore issues of fairness in machine learning (ML). Unlike in previous psets, where we were implementing algorithms from scratch, this pset will rely more heavily on sklearn. Unless explicitly noted otherwise, you are allowed (encouraged!) to make good use of this wonderful library.

This problem set will also rely on the resources provided by the [folktables](#) project. Before getting started, take some time to understand what folktables is about and how it can be used to benchmark ML algorithms in social science. If you don't understand the basics, this pset will be very challenging!!

Some resources you might find useful:

[Folktables paper](#)

[Folktables video](#)

[PUMS_Data_Dictionary_2018](#)

[Fairness in ML](#)

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import folktables
import seaborn as sns
```

Part I: Setup of prediction task

1.1 Define the income prediction task

For this first question we are providing the code. Run it to download the data you'll be using throughout this problem set.

Begin by explaining what this prediction task is about and what is this code accomplishing:

- Where do the data come from
- What is the sample?
- What are we trying to predict?
- What are the features that we will be using?
- What is the "group" feature about?

In []:

```

from folktables import ACSDataSource, generate_categories

def adult_filter(data):
    """Mimic the filters in place for Adult data.

    Adult documentation notes: Extraction was done by Barry Becker from
    the 1994 Census database. A set of reasonably clean records was extracted
    using the following conditions:
    ((AAGE>16) && (AGI>100) && (AFNLWGT>1)&& (HRSWK>0))
    """
    df = data
    df = df[df['AGEP'] > 16]
    df = df[df['PINCP'] > 100]
    df = df[df['WKHP'] > 0]
    df = df[df['PWGTP'] >= 1]
    return df

ACSIIncome = folktables.BasicProblem(
    features=[
        'AGEP',
        'COW',
        'SCHL',
        'MAR',
        'POBP',
        'RELP',
        'WKHP',
        'SEX',
        'RAC1P',
    ],
    target='PINCP',
    target_transform = lambda x: x > 50000,
    group='RAC1P',
    preprocess=adult_filter,
    postprocess=lambda x: np.nan_to_num(x, -1),
)

data_source = ACSDataSource(survey_year='2018', horizon='1-Year', survey='person')
acs_data = data_source.get_data(states=["CA"], download=True)
definition_df = data_source.get_definitions(download=True)
categories = generate_categories(features=ACSIIncome.features, definition_df=definition_
features, target, group = ACSIIncome.df_to_pandas(acs_data)

feature_names = ACSIIncome.features
target_name = ACSIIncome.target
group_name = ACSIIncome.group

```

- Where do the data come from?
 - This data comes from Folktables, a python package that provides access to US census datasets. The data source used in this function comes from 2018 american community survey specifically for the state of California.
- What is the sample?
 - The data sample used in this function comes from 2018 american community survey specifically for the state of California. The adult_filter function is filtering the data for Age > 16, Income > 100, Hours worked per week >0 and a person weighting variable >= 1.

- What are we trying to predict?
 - The target variable here is 'PINCP' which is total person's income. There is a transform applied to the target variable. We are trying to predict whether an individual's income is above \$50,000.
- What are the features that we will be using?
 - The features we are using are:
 - 'AGEP' = Age
 - 'COW' = Class of worker
 - 'SCHL' = Educational attainment
 - 'MAR' = Marital status
 - 'POBP' = Place of birth
 - 'RELP' = relationship
 - 'WKHP' = Hours worked per week (past year)
 - 'SEX' = sex
 - 'RAC1P' = race
- What is the "group" feature about?
 - The group option allows for groupings of the data based on a specific feature, like 'RAC1P': that could allow for the evaluation of a given model's violation of equality of opportunity.

1.2 Split into train and test

Split your data into 80%-20% train and test splits. Please use `sklearn.model_selection.train_test_split` and set the `random_state` parameter equal to 0.

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test, group_train, group_test = train_test_split(
    features, target, group, test_size=0.2, random_state=0)
```

1.3 Explore the data and set appropriate data types

Create 2-3 figures and tables that explore the training data, and tell us what insights you can draw from those figures. Which features are best represented as numerical data types? Which features are best represented as categorical data types? Set each feature to its desired data type in both train and test splits. Make sure that the target variable in both splits is numeric before moving ahead.

```
In [ ]: # Set numerical variables

X_train['WKHP'] = X_train['WKHP'].astype('float64')
X_train['AGEP'] = X_train['AGEP'].astype('float64')

X_test['WKHP'] = X_test['WKHP'].astype('float64')
X_test['AGEP'] = X_test['AGEP'].astype('float64')

# Set categorical variables
X_train['COW'] = X_train['COW'].astype('int').astype('category')
X_train['SCHL'] = X_train['SCHL'].astype('int').astype('category')
X_train['MAR'] = X_train['MAR'].astype('int').astype('category')
```

```

X_train['POBP'] = X_train['POBP'].astype('int').astype('category')
X_train['SEX'] = X_train['SEX'].astype('int').astype('category')
X_train['RAC1P'] = X_train['RAC1P'].astype('int').astype('category')

X_test['COW'] = X_test['COW'].astype('int').astype('category')
X_test['SCHL'] = X_test['SCHL'].astype('int').astype('category')
X_test['MAR'] = X_test['MAR'].astype('int').astype('category')
X_test['POBP'] = X_test['POBP'].astype('int').astype('category')
X_test['SEX'] = X_test['SEX'].astype('int').astype('category')
X_test['RAC1P'] = X_test['RAC1P'].astype('int').astype('category')

# Convert target true and false to binary
y_train['PINCP'] = y_train['PINCP'].astype('int')
y_test['PINCP'] = y_test['PINCP'].astype('int')

```

In []:

```

# Figure 1: Hours worked per week
sns.histplot(X_train['WKHP'])
plt.vlines(X_train['WKHP'].mean(), 0, 75000, linestyle = 'dashed', color = 'black', label = 'Mean hours worked per week')
plt.xlabel('Usual hours worked per week past 12 months')
plt.ylabel('Individuals')
plt.title('Histogram of hours worked per week for training data')
plt.legend()
sns.despine()

```

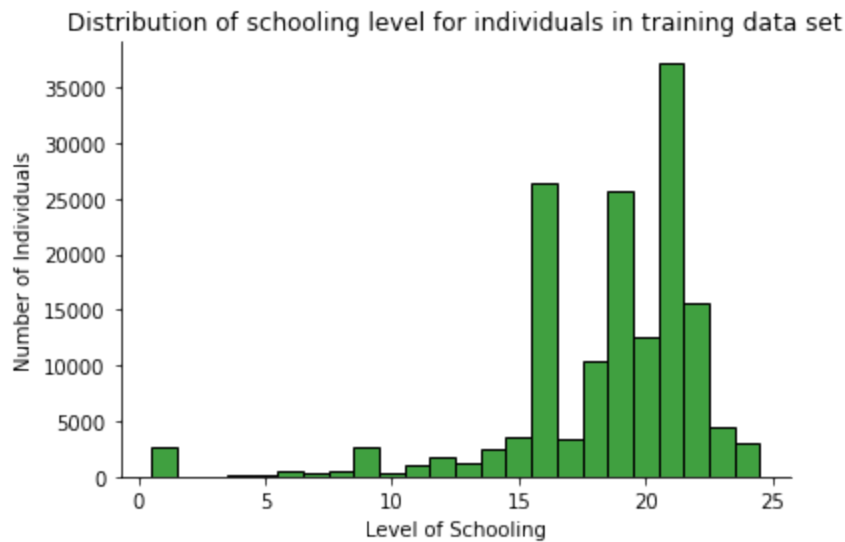


In []:

```

# Figure 2:
sns.histplot(X_train['SCHL'], color = 'green')
plt.xlabel('Level of Schooling')
plt.ylabel('Number of Individuals')
plt.title('Distribution of schooling level for individuals in training data set')
sns.despine()

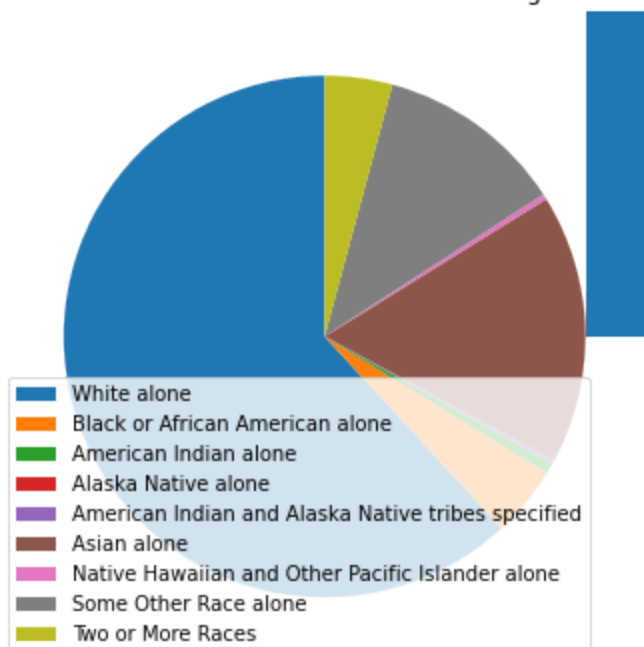
```



In []:

```
# Figure 3: Race distribution pie chart for training data
plt.figure(figsize=(6,6))
race_dist = plt.hist(X_train['RAC1P'], bins = 9)
labels_RAC1P = ['White alone',
                'Black or African American alone',
                'American Indian alone',
                'Alaska Native alone',
                'American Indian and Alaska Native tribes specified',
                'Asian alone',
                'Native Hawaiian and Other Pacific Islander alone',
                'Some Other Race alone',
                'Two or More Races']
# plt.pie(race_dist[0], labels=labels_RAC1P, autopct='%1.1f%%')
patches, texts = plt.pie(race_dist[0], startangle=90)
plt.legend(patches, labels_RAC1P, loc="lower left")
plt.title('Distribution of race for individuals in training data set')
plt.show()
```

Distribution of race for individuals in training data set



1. Which features are best represented as numerical data types?

- age (AGEP) and usual hours worked per week (WKHP)

1. Which features are best represented as categorical data types?

- class of worker (COW), marital status (MAR), sex (SEX), racial code (RAC1P), place of birth (POBP), educational attainment (SCHL)

Part II: ML pipeline for a Classification Tree

Next, you'll build an ML pipeline using sklearn. Take some time to familiarize yourself with [sklearn.pipeline.Pipeline](#) before proceeding.

2.1 Pre-processing of features

The first step of the pipeline will standardize the numeric features and one-hot encode the categorical features. Write the code for this preprocessing step below.

Hint: Take a look at sklearn's ColumnTransformer, OneHotEncoder and StandardScaler for this task.

In []:

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Define numeric features and transform them using standard scaler
numeric_features = ['WKHP', 'AGEP']
numeric_transformer = Pipeline(
    steps=[("scaler", StandardScaler())]
)

# Define categorical features and encode them using one hot encoder
categorical_features = ['COW', 'SCHL', 'MAR', 'POBP', 'RELP', 'SEX', 'RAC1P']
categorical_transformer = Pipeline(
    steps=[("encoder", OneHotEncoder(handle_unknown='ignore'))]
)

# Define pre-processor for later on
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ]
)

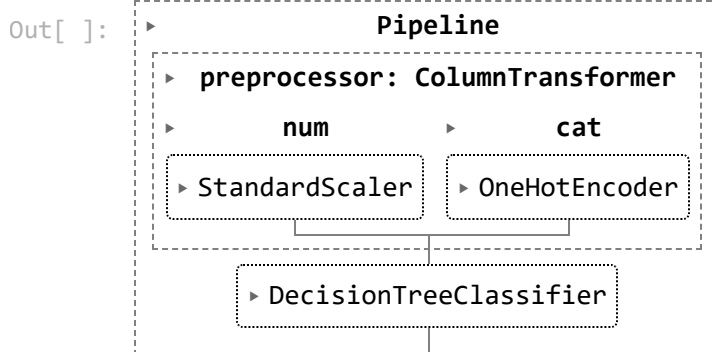
# Preprocess the data
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)
```

2.2 Use your pre-processed data to fit a basic DecisionTreeClassifier

Now, bring together the pre-processing step with a Decision Tree Classifier in a pipeline. For now, use the default values; we'll take care of hyperparameter optimization later. Fit the pipeline on the training data. How deep is the resulting tree? How many leafs are in the tree? Print the 10 variables with highest feature importance and tell us what those importances mean in plain English. Plot the top 5 levels of the fitted decision tree. Make sure that the nodes are appropriately labeled. Comment on anything that you find interesting.

```
In [ ]: from sklearn.tree import DecisionTreeClassifier
        from sklearn import tree

        # Fit model using pipeline
        clf = Pipeline(steps=[("preprocessor", preprocessor), ("classifier", DecisionTreeClassifier)])
        clf.fit(X_train, y_train)
```



```
In [ ]: # 1. Determine the depth of the tree
        tree_depth = clf['classifier'].get_depth()
        print("Depth of the tree:", tree_depth)

        # 2. Count the number of leafs in the tree
        leaf_count = clf['classifier'].get_n_leaves()
        print("Number of leaf nodes:", leaf_count)

        # 3. 10 variables with highest feature importance
        # Get feature importances
        feature_importance = clf['classifier'].feature_importances_
        top_features = np.argsort(feature_importance)[::-1][:10]

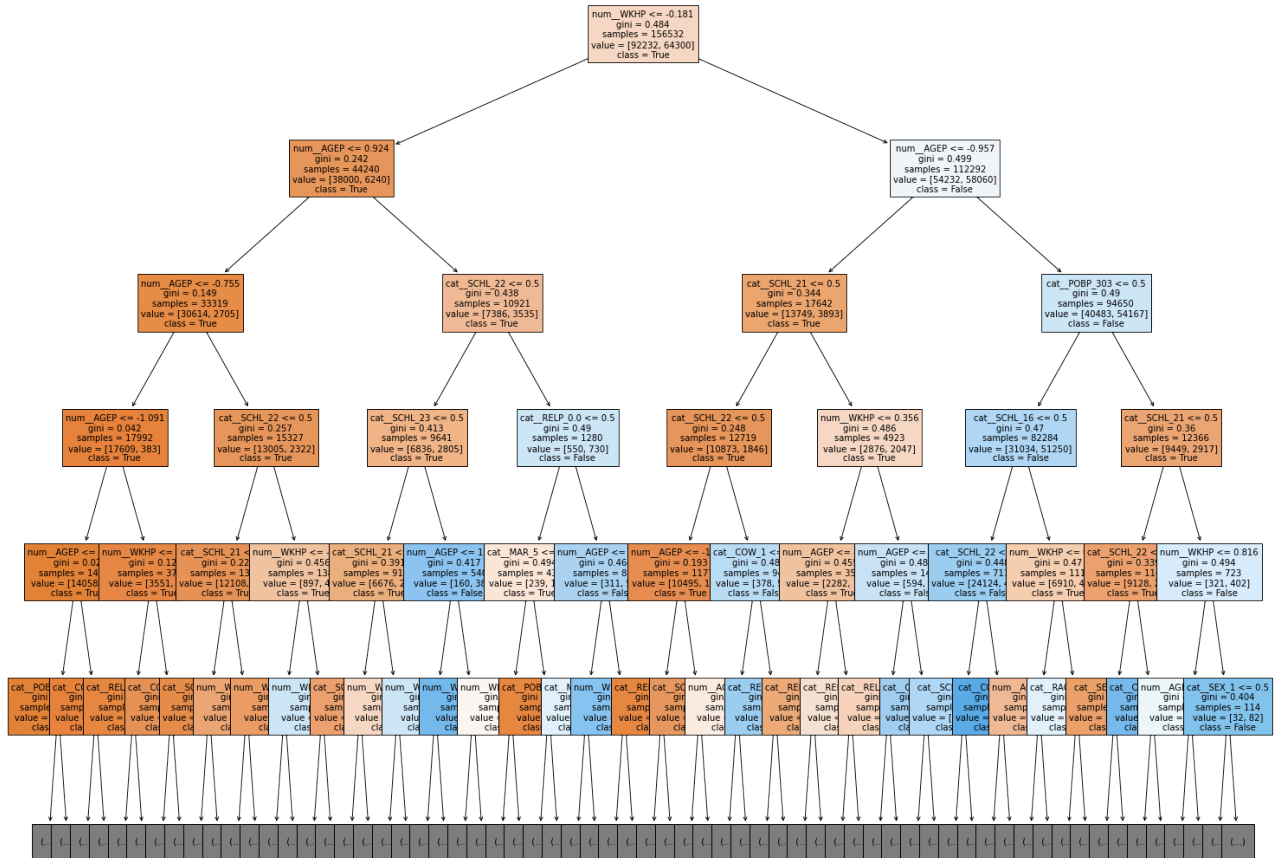
        # Get names of transformed features
        new_feature_names = preprocessor.get_feature_names_out()

        print("Top 10 variables with highest feature importance:")
        for idx in top_features:
            print(f"{new_feature_names[idx]}: {feature_importance[idx]}")

        # 4. Plot the top 5 levels of the fitted decision tree
        fig = plt.figure(figsize=(25,20))
        tree.plot_tree(clf['classifier'], max_depth=5, feature_names = new_feature_names, class_
        plt.show()
```

Depth of the tree: 88
 Number of leaf nodes: 37889
 Top 10 variables with highest feature importance:
 num__AGEP: 0.2336718081551264

num_WKHP: 0.20110720194768678
 cat_POBP_303: 0.04856997464276286
 cat_SCHL_21: 0.031870444669552245
 cat_SCHL_22: 0.027116010059415465
 cat_SCHL_16: 0.024497287736344332
 cat_SCHL_23: 0.018335039577390334
 cat_POBP_6: 0.016713378724138934
 cat_COW_1: 0.015924649877269135
 cat_REL_P_0.0: 0.01590516959562802



ANSWERS

1. The variables with the highest feature importance are:

- num_AGE: 0.2336718081551264
- num_WKHP: 0.20110720194768678
- cat_POBP_303: 0.04856997464276286
- cat_SCHL_21: 0.031870444669552245
- cat_SCHL_22: 0.027116010059415465
- cat_SCHL_16: 0.024497287736344332
- cat_SCHL_23: 0.018335039577390334
- cat_POBP_6: 0.016713378724138934
- cat_COW_1: 0.015924649877269135
- cat_REL_P_0.0: 0.01590516959562802

2. Feature importance for a decision tree represents the relative importance of each feature in making predictions in the decision tree. The feature importance is based off of how much each

feature reduces the Gini impurity in the *tree node*. Higher importance indicates that sub-feature has a greater effect on reduction of impurity and hence a stronger influence on the tree's decision. For our decision tree, it appears that age and hours worked per week have great importance in predicting the target. The schooling level and place of birth also seem to have high predictivity, with race (whether you're white) is also important in predicting the target.

2.3 Estimate AUC using K-Fold CV

Calculate the 5-fold cross-validated AUC-ROC for this simple tree pipeline. Report the score for each fold as well as the average across all folds.

Hint 1: `sklearn.model_selection.cross_val_score` is your friend. Setting the verbose option to 3 is useful.

```
In [ ]: from sklearn.model_selection import cross_val_score

# Calculate cross-validated AUC-ROC scores
tree_clf = Pipeline(steps=[("preprocessor", preprocessor), ("classifier", DecisionTreeC
auc_roc_scores = cross_val_score(tree_clf, X_train, y_train, cv=5, scoring='roc_auc', v

# Calculate the mean AUC-ROC score
mean_auc_roc_score = auc_roc_scores.mean()

[CV] END ..... score: (test=0.736) total time= 16.2s
[CV] END ..... score: (test=0.739) total time= 16.4s
[CV] END ..... score: (test=0.736) total time= 15.6s
[CV] END ..... score: (test=0.737) total time= 15.8s
[CV] END ..... score: (test=0.737) total time= 15.3s
```

```
In [ ]: # Report scores
print('AUC-ROC score for each fold:' + str(auc_roc_scores))
print('Mean 5-fol CV AUC-ROC: ' + str(mean_auc_roc_score))

AUC-ROC score for each fold:[0.73634837 0.73854879 0.73581141 0.73735245 0.73675995]
Mean 5-fol CV AUC-ROC: 0.7369641943576709
```

2.4 Estimating AUC using nested CV

Let's try to improve on the results of the decision tree by tweaking its hyperparameters. Since you already are an expert using nested CV, this is going to be a walk in the park! Compute the nested cv AUC-ROC of a decision tree pipeline. Use 5 folds for the inner loop and 3 folds for the outer loop. Include different values for "max_depth", "max_features" and "max_leave_nodes" in your hyperparameter grid. Report the inner and outer fold scores for all combinations of hyperparams and interpret the results. Report the best hyperparameters used in each outer fold. How stable are the results?

What else do you notice in these results? To what extent do you think specific hyperparameters might lead to overfitting? How is run-time impacted by your choice of hyperparameters? Is hyperparameter optimization worth the trouble?

Hint 1: The pipeline remains the same. You can access the list of all the hyperparams in your pipeline with `pipeline.get_params_keys()`. Notice the naming convention.

Hint 2: `GridSearchCV` and `cross_val_score` are your friends-- especially if you run them with `verbose = 3`.

Hint 3: This gets computationally expensive quickly. Be intelligent about the hyperparameter values you include in the grid and the number of outer and inner folds you use (we use 5 inner folds and 3 outer folds).

In []:

```
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.metrics import roc_auc_score

# Define hyperparameter grid
tot_n_features = len(new_feature_names)
param_grid = {
    'classifier__max_depth': [25, 50, 75],
    'classifier__max_features': [tot_n_features, 'sqrt', 'log2'],
    'classifier__max_leaf_nodes': [1000, 1500, 2000]
}

# Initialize inner/outer cross-validation folds
outer_cv_folds = 3
inner_cv_folds = 5

# Initialize lists to store results
outer_scores = []
best_params_list = []

# Perform 5-fold cross-validation
kf = KFold(n_splits=outer_cv_folds, shuffle=True, random_state=0)

# Outer cross-validation loop
for train_idx, test_idx in kf.split(X_train):
    X_train_fold, X_test_fold = X_train.iloc[train_idx], X_train.iloc[test_idx]
    y_train_fold, y_test_fold = y_train.iloc[train_idx], y_train.iloc[test_idx]

    # Inner cross-validation for hyperparameter tuning
    inner_cv = GridSearchCV(tree_clf, param_grid, cv=inner_cv_folds, scoring='roc_auc',
                           inner_cv.fit(X_train_fold, y_train_fold)

    # Best hyperparameters for each outer fold
    best_params = inner_cv.best_params_
    best_params_list.append(best_params)

    # Note: Inner fold CV score for each combination of hyperparameters is printed using
    # Use best model to calculate outer fold score
    best_model = inner_cv.best_estimator_
    y_pred = best_model.predict_proba(X_test_fold)[: , 1]
    outer_score = roc_auc_score(y_test_fold, y_pred)
    outer_scores.append(outer_score)
```

Fitting 5 folds for each of 27 candidates, totalling 135 fits
 [CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000; score=0.852 total time= 3.7s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.857 total time= 3.6s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.858 total time= 3.6s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.860 total time= 3.8s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.855 total time= 3.7s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.845 total time= 4.5s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.850 total time= 4.5s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.850 total time= 4.7s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.851 total time= 4.6s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.848 total time= 4.4s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.836 total time= 5.3s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.841 total time= 5.2s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.844 total time= 5.5s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.844 total time= 5.3s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.839 total time= 5.2s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.819 total time= 0.4s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.837 total time= 0.3s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.826 total time= 0.3s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.827 total time= 0.3s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.823 total time= 0.3s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.817 total time= 0.4s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.837 total time= 0.4s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.825 total time= 0.4s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.824 total time= 0.4s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.823 total time= 0.4s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.816 total time= 0.5s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.834 total time= 0.4s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.823 total time= 0.5s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.822 total time= 0.5s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.821 total time= 0.4s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.784 total time= 0.2s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.788 total time= 0.2s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.795 total time= 0.2s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.791 total time= 0.2s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.786 total time= 0.2s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.783 total time= 0.3s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.786 total time= 0.3s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.796 total time= 0.3s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.789 total time= 0.3s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.785 total time= 0.3s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.781 total time= 0.3s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.787 total time= 0.4s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.798 total time= 0.4s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.787 total time= 0.3s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.783 total time= 0.4s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.852 total time= 3.8s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.857 total time= 3.7s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.857 total time= 3.8s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.860 total time= 3.9s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.855 total time= 3.9s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.846 total time= 4.8s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.850 total time= 5.1s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.850 total time= 5.5s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.851 total time= 5.4s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.848 total time= 5.5s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.837 total time= 6.5s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.841 total time= 6.6s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.844 total time= 6.7s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.845 total time= 6.5s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.840 total time= 6.5s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.820 total time= 0.4s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.838 total time= 0.4s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.833 total time= 0.4s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.825 total time= 0.4s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.824 total time= 0.4s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.821 total time= 0.6s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.837 total time= 0.5s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.831 total time= 0.5s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.826 total time= 0.5s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.827 total time= 0.5s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.820 total time= 0.7s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.836 total time= 0.6s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.829 total time= 0.6s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.822 total time= 0.6s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.824 total time= 0.6s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.774 total time= 0.4s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.788 total time= 0.3s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.795 total time= 0.3s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.812 total time= 0.3s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.796 total time= 0.3s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.786 total time= 0.5s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.789 total time= 0.4s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.794 total time= 0.4s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.814 total time= 0.4s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.795 total time= 0.4s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.786 total time= 0.6s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.792 total time= 0.5s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.793 total time= 0.5s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.813 total time= 0.5s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.799 total time= 0.5s

[CV 1/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.852 total time= 4.3s

[CV 2/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.857 total time= 4.4s

[CV 3/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.857 total time= 4.5s

[CV 4/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.860 total time= 4.8s

[CV 5/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.855 total time= 5.1s

[CV 1/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.846 total time= 6.9s

[CV 2/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.850 total time= 6.3s

[CV 3/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.850 total time= 5.5s

[CV 4/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.851 total time= 5.3s

[CV 5/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.848 total time= 5.2s

[CV 1/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.837 total time= 6.7s

[CV 2/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.841 total time= 6.7s

[CV 3/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.844 total time= 6.8s

[CV 4/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.845 total time= 6.4s

[CV 5/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.840 total time= 6.3s

[CV 1/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.820 total time= 0.4s

[CV 2/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.838 total time= 0.4s

[CV 3/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.833 total time= 0.4s

[CV 4/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.825 total time= 0.4s

[CV 5/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.824 total time= 0.4s

[CV 1/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.821 total time= 0.6s

[CV 2/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.838 total time= 0.5s

[CV 3/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.831 total time= 0.6s

[CV 4/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.824 total time= 0.5s

[CV 5/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.827 total time= 0.5s

[CV 1/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.819 total time= 0.7s

[CV 2/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.836 total time= 0.6s

[CV 3/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.830 total time= 0.6s

[CV 4/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.820 total time= 0.6s

[CV 5/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.826 total time= 0.6s

[CV 1/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.787 total time= 0.4s

```

[CV 2/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.792 total time= 0.3s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.795 total time= 0.3s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.809 total time= 0.3s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.792 total time= 0.3s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.792 total time= 0.5s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.794 total time= 0.4s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.795 total time= 0.4s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.814 total time= 0.4s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.793 total time= 0.4s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.791 total time= 0.6s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.794 total time= 0.5s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.794 total time= 0.5s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.815 total time= 0.5s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.800 total time= 0.4s
Fitting 5 folds for each of 27 candidates, totalling 135 fits
[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.853 total time= 4.1s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.859 total time= 4.0s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.857 total time= 3.9s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.860 total time= 3.9s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.859 total time= 3.8s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.845 total time= 4.7s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.850 total time= 4.9s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.850 total time= 4.9s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.852 total time= 4.8s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.850 total time= 4.7s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.836 total time= 5.4s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.841 total time= 8.4s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.842 total time= 8.7s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.843 total time= 7.6s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.842 total time= 7.6s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le

```

af_nodes=1000;; score=0.818 total time= 0.5s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.829 total time= 0.5s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.822 total time= 0.5s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.817 total time= 0.5s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.827 total time= 0.5s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.820 total time= 0.6s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.831 total time= 0.6s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.820 total time= 0.5s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.818 total time= 0.5s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.825 total time= 0.5s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.819 total time= 0.6s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.829 total time= 0.6s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.818 total time= 0.5s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.817 total time= 0.6s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.821 total time= 0.6s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.766 total time= 0.3s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.809 total time= 0.3s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.797 total time= 0.3s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.807 total time= 0.3s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.803 total time= 0.3s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.770 total time= 0.3s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.809 total time= 0.3s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.797 total time= 0.3s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.807 total time= 0.3s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.804 total time= 0.4s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.769 total time= 0.5s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.807 total time= 0.4s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.797 total time= 0.4s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.806 total time= 0.4s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.803 total time= 0.5s
[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea


```
f_nodes=1000;; score=0.853 total time= 4.1s
[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.860 total time= 4.2s
[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.857 total time= 4.2s
[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.860 total time= 4.2s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.859 total time= 4.2s
[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.845 total time= 5.1s
[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.850 total time= 5.2s
[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.850 total time= 4.9s
[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.851 total time= 5.0s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.850 total time= 4.9s
[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.837 total time= 5.8s
[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.843 total time= 7.1s
[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.842 total time= 7.4s
[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.842 total time= 6.2s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.843 total time= 7.2s
[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.824 total time= 0.7s
[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.827 total time= 0.5s
[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.834 total time= 0.5s
[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.819 total time= 0.5s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.824 total time= 0.5s
[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.823 total time= 0.6s
[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.828 total time= 0.6s
[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.832 total time= 0.6s
[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.821 total time= 0.7s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.825 total time= 0.6s
[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.825 total time= 0.7s
[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.826 total time= 0.8s
[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.830 total time= 0.7s
[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.819 total time= 0.7s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.824 total time= 0.7s
[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
```

```
af_nodes=1000;; score=0.784 total time= 0.4s
[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.818 total time= 0.4s
[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.782 total time= 0.4s
[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.805 total time= 0.4s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.805 total time= 0.4s
[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.783 total time= 0.6s
[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.816 total time= 0.5s
[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.783 total time= 0.5s
[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.804 total time= 0.5s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.802 total time= 0.5s
[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.785 total time= 0.7s
[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.816 total time= 0.6s
[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.796 total time= 0.6s
[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.802 total time= 0.6s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.801 total time= 0.6s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.853 total time= 5.2s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.860 total time= 5.2s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.857 total time= 5.1s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.860 total time= 4.9s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.859 total time= 5.2s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.845 total time= 6.2s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.850 total time= 6.0s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.850 total time= 6.1s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.851 total time= 6.2s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.850 total time= 6.3s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.837 total time= 10.1s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.843 total time= 8.2s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.842 total time= 7.4s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.842 total time= 8.6s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_lea
f_nodes=2000;; score=0.843 total time= 8.9s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
```

```
af_nodes=1000;; score=0.824 total time= 0.6s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.827 total time= 0.6s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.834 total time= 0.5s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.819 total time= 0.5s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000;; score=0.824 total time= 0.5s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.824 total time= 0.6s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.828 total time= 0.7s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.834 total time= 0.6s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.822 total time= 0.7s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.825 total time= 0.8s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.825 total time= 0.8s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.827 total time= 0.8s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.833 total time= 0.8s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.820 total time= 0.8s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_le
af_nodes=2000;; score=0.823 total time= 0.9s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.789 total time= 0.6s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.817 total time= 0.5s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.782 total time= 0.5s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.805 total time= 0.6s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.808 total time= 0.5s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.793 total time= 0.6s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.814 total time= 0.6s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.784 total time= 0.7s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.804 total time= 0.6s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.809 total time= 0.6s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.792 total time= 0.7s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.812 total time= 0.7s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.794 total time= 0.8s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.809 total time= 0.8s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.807 total time= 0.6s
Fitting 5 folds for each of 27 candidates, totalling 135 fits
```

[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.852 total time= 4.9s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.856 total time= 4.5s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.857 total time= 4.5s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.852 total time= 4.0s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.857 total time= 4.0s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.845 total time= 4.9s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.848 total time= 4.8s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.849 total time= 4.8s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.842 total time= 4.8s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.849 total time= 4.8s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.837 total time= 5.5s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.839 total time= 5.3s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.840 total time= 5.5s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.835 total time= 5.6s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.840 total time= 5.6s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.834 total time= 0.4s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.809 total time= 0.4s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.841 total time= 0.4s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.826 total time= 0.4s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.843 total time= 0.4s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.832 total time= 0.5s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.807 total time= 0.5s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.838 total time= 0.4s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.824 total time= 0.5s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.843 total time= 0.5s
[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.831 total time= 0.5s
[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.805 total time= 0.6s
[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.836 total time= 0.5s
[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.823 total time= 0.6s
[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.841 total time= 0.5s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.773 total time= 0.3s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.821 total time= 0.3s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.773 total time= 0.3s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.786 total time= 0.3s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.804 total time= 0.3s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.773 total time= 0.3s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.820 total time= 0.4s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.773 total time= 0.4s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.784 total time= 0.4s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.802 total time= 0.3s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.771 total time= 0.4s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.819 total time= 0.4s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.774 total time= 0.4s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.783 total time= 0.4s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.801 total time= 0.4s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.852 total time= 4.1s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.856 total time= 4.1s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.857 total time= 3.9s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.852 total time= 3.8s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.856 total time= 3.9s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.846 total time= 4.9s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.849 total time= 4.9s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.849 total time= 4.9s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.843 total time= 4.9s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.849 total time= 5.1s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.839 total time= 6.5s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.840 total time= 5.8s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.841 total time= 6.3s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.835 total time= 5.7s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.840 total time= 5.8s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.838 total time= 0.4s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.812 total time= 0.5s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.838 total time= 0.5s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.830 total time= 0.5s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.845 total time= 0.5s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.837 total time= 0.5s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.812 total time= 0.6s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.837 total time= 0.5s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.830 total time= 0.5s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.843 total time= 0.5s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.834 total time= 0.6s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.811 total time= 0.6s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.834 total time= 0.6s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.829 total time= 0.6s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.840 total time= 0.6s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.790 total time= 0.3s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.816 total time= 0.3s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.798 total time= 0.3s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.804 total time= 0.3s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.804 total time= 0.4s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.789 total time= 0.4s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.814 total time= 0.4s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.800 total time= 0.4s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.807 total time= 0.4s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.806 total time= 0.5s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.795 total time= 0.5s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.812 total time= 0.4s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.801 total time= 0.5s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.806 total time= 0.6s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=2000;; score=0.805 total time= 0.6s

[CV 1/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.852 total time= 4.4s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.856 total time= 3.9s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.857 total time= 4.0s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.852 total time= 4.0s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.856 total time= 3.9s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.846 total time= 5.0s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.849 total time= 5.0s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.849 total time= 4.9s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.843 total time= 5.3s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.849 total time= 5.3s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.839 total time= 5.7s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.840 total time= 6.0s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.841 total time= 5.7s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.835 total time= 5.7s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=287, classifier__max_leaf_nodes=2000;; score=0.840 total time= 5.7s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.838 total time= 0.4s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.812 total time= 0.5s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.838 total time= 0.4s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.830 total time= 0.4s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.845 total time= 0.5s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.837 total time= 0.5s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.813 total time= 0.6s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.836 total time= 0.6s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.830 total time= 0.5s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.843 total time= 0.5s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.836 total time= 0.7s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.812 total time= 0.8s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.834 total time= 0.7s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.829 total time= 0.7s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=sqrt, classifier__max_leaf_nodes=2000;; score=0.841 total time= 0.7s

```
[CV 1/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.789 total time= 0.3s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.822 total time= 0.3s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.798 total time= 0.3s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.804 total time= 0.3s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1000;; score=0.804 total time= 0.3s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.790 total time= 0.4s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.821 total time= 0.4s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.801 total time= 0.4s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.807 total time= 0.4s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=1500;; score=0.803 total time= 0.4s
[CV 1/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.791 total time= 0.4s
[CV 2/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.820 total time= 0.4s
[CV 3/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.800 total time= 0.5s
[CV 4/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.806 total time= 0.5s
[CV 5/5] END classifier__max_depth=75, classifier__max_features=log2, classifier__max_le
af_nodes=2000;; score=0.806 total time= 0.5s
```

In []:

```
# 1. Report outer fold scores for best model (optimized hyperparameters)
print("Outer fold scores:", outer_scores)
print('Mean outer fold score: ', np.mean(outer_scores))

# 2. Report best hyperparameters used in each outer fold
print("Best hyperparameters for each outer fold:")
for i, params in enumerate(best_params_list):
    print(f"Fold {i+1}: {params}")
```

Outer fold scores: [0.8615225351105268, 0.8599465017648833, 0.8602078085026861]

Mean outer fold score:

Best hyperparameters for each outer fold:

Fold 1: {'classifier__max_depth': 50, 'classifier__max_features': 287, 'classifier__max_leaf_nodes': 1000}

Fold 2: {'classifier__max_depth': 50, 'classifier__max_features': 287, 'classifier__max_leaf_nodes': 1000}

Fold 3: {'classifier__max_depth': 25, 'classifier__max_features': 287, 'classifier__max_leaf_nodes': 1000}

- Report the inner and outer fold scores for all combinations of hyperparams and interpret the results.
 - The inner fold scores are printed in the model fitting procedure and the outer fold scores are printed above.
- Report the best hyperparameters used in each outer fold. How stable are the results?

- The best hyperparameters for each outer fold are printed above. Analyzing the results, they are fairly stable with the max depth = 50, max_features = 287, and max_leaf_nodes = 1000 being the optimal hyperparameters. On one of the outer folds the max_depth parameter was different from the other 2 folds (25 vs 50), otherwise for the other hyperparameters the optimal values were consistent across the 3-folds.
- What else do you notice in these results? To what extent do you think specific hyperparameters might lead to overfitting? How is run-time impacted by your choice of hyperparameters? Is hyperparameter optimization worth the trouble?
 - I notice that in general model performance worsened when the max number of features was less than the total number of features.
 - Generally, by increasing the dimensionality of the data set (ie: by increasing the max number of features that are considered in a split) we are increasing the complexity of the model which can lead to overfitting. Therefore, by reducing the max_features hyperparameter we may prevent overfitting. In the case of the current approach, since we are only using only the training data set for nested CV hyperparameter tuning, it would be a good idea to evaluate model performance on the held out test data using regular CV for hyperparameter tuning and then using that model to fit the x_test and compare the predicted with the actual.
 - The run time for a single fit is significantly increased by increasing the max_features hyperparameter. It is also impacted by the max_depth of the tree (increasing time with increasing depth). The max leaf node hyperparameter with the current range of values seems to have less of an impact on run time, increasing the run time for increasing hyperparameter value.
 - Here we ran just 3 x 3 hyperparameter search grid which leads to a total of 27 candidates totalling 135 fits for 5 fold inner-CV. This was ran 3 times for a 3-fold outer CV fit. This is was more computationally expensive than just running the model a couple of times tweaking the parameters but not so long that it greatly outweighed trouble of optimization. I ran this on my workstation and the total run time was 15 minutes, which is not bad but definitely not ideal. We can reduce the run-time next time by reudcing the number of inner CV folds and reducing the parameter search space.

Part III: ML pipeline for a random forest

3.1 Bare bones random forest

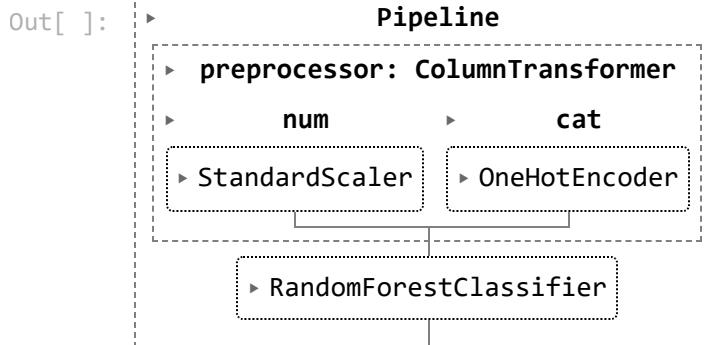
Just as you did for the classification tree, start by training a bare bones random forest using your training data. We will take care of the hyperparameter optimization later, but for the time being, use a maximum tree depth of 15. Hopefully is it clear by now why this is a good idea, since we are going to be training a bunch of trees. You should continue to use the same preprocessing step as before.

How many trees are in the forest? Which are the most important features? Compute the fraction of the trees that are splitting on age at their root node. Create a plot showing the 10 most important features and compare them to the results you obtained before. Explain the meaning of variable importance in the case of a random forest classifier.

```
In [ ]: from sklearn.ensemble import RandomForestClassifier

# Define Random Forest classifier
rf_clf = Pipeline(steps=[("preprocessor", preprocessor), ("classifier", RandomForestClassifier)]

# Train the classifier using the preprocessing step
rf_clf.fit(X_train, y_train)
```



```
In [ ]: # 1. Get the number of trees in the forest
num_trees = len(rf_clf['classifier'].estimators_)
print("Number of trees in the forest:", num_trees)

# 2. Which are the most important features?
important_features = rf_clf['classifier'].feature_importances_
top_feature_idx = np.argsort(important_features)[::-1][:10]
top_10_features = [new_feature_names[i] for i in top_feature_idx]

print("Top 10 most important features:")
for feature in top_10_features:
    print(feature)

# 3. Compute the fraction of trees that are splitting on age at their root node
age_idx = np.where(new_feature_names == 'num__AGEP')[0][0] # find AGEP feature position
root_node = 0

# Iterate through trees to find what feature is being split on root node
age_root_node_count = sum([1 for tree in rf_clf['classifier'].estimators_ if tree.tree_
    fraction_age_root_node = age_root_node_count / num_trees
print("Fraction of trees splitting on age at their root node:", fraction_age_root_node)

# 4. Plot the feature importances of the top 10 features
plt.bar(range(10), important_features[top_feature_idx], tick_label=top_10_features)
plt.xlabel("Features")
plt.ylabel("Mean decrease in impurity ")
plt.title("Top 10 Important Features")
plt.xticks(rotation=45, ha="right")
sns.despine()
plt.show()
```

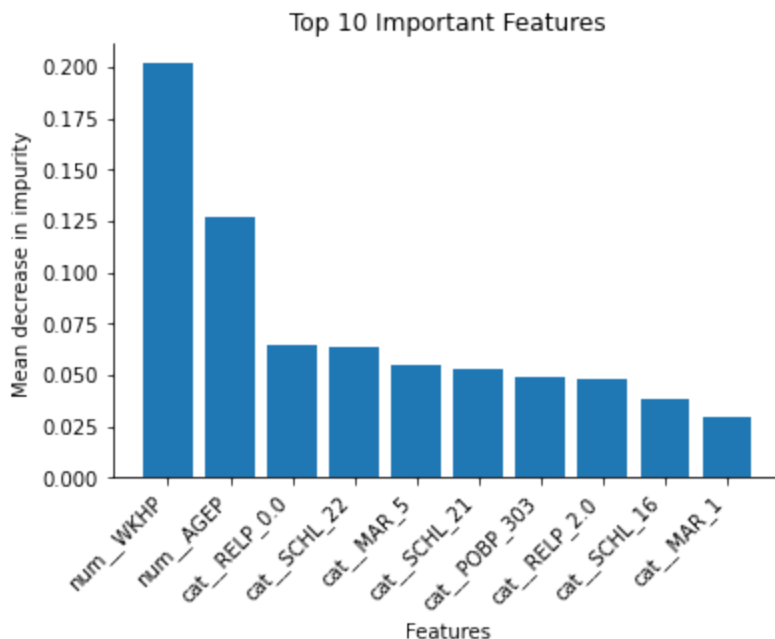
Number of trees in the forest: 100

Top 10 most important features:

```
num__WKHP
num__AGEP
cat__RELP_0.0
cat__SCHL_22
```

cat__MAR_5
 cat__SCHL_21
 cat__POBP_303
 cat__RELP_2.0
 cat__SCHL_16
 cat__MAR_1

Fraction of trees splitting on age at their root node: 0.09



1. Compare top 10 important features to the results you obtained before:

- Here in our RF classifier we find that **numWKHP**, **numAGEP**, **catRELP_0.0**, **catSCHL_22**, **catMAR_5**, **catSCHL_21**, **catPOBP_303**, **catRELP_2.0**, **catSCHL_16** and **catMAR_1** are the top 10 most important features. Comparing this to our DT classifier we find that, it consistently ranks age and number of hours worked per week as the most important features to be considered when deciding to split. From there on the features change a bit with relationship to reference person and marital status being included as top features vs more schooling and employment type being considered for the decision tree.

1. Explain the meaning of variable importance in the case of a random forest classifier.

- Feature importance for a Random Forest classifier measures the contribution of each feature to the overall predictive performance of the model. Feature importance is computed as the *mean* reduction in Gini impurity obtained by splitting on the feature across ALL trees in the forest. Features with higher importance values indicate higher predictive power of the target variable. This definition for RF is generally a more robust measure of feature importance.

3.2 Estimating AUC using K-fold CV

Calculate the 5-fold cross-validated AUC-ROC for the random forests pipeline. Report the score for each fold as well as the average across all folds.

Hint 1: `sklearn.model_selection.cross_val_score` is your friend. Setting the verbose option to 3 is useful.

```
In [ ]: # Calculate 5-fold cross-validated AUC-ROC
cv_scores = cross_val_score(rf_clf, X_train, y_train.values.ravel(), cv=5, scoring='roc')

# Print the score for each fold
print("AUC-ROC scores for each fold:", cv_scores)

# Calculate the average AUC-ROC across all folds
mean_auc_roc = cv_scores.mean()
print("Mean AUC-ROC across all folds:", mean_auc_roc)
```

```
[CV] END ..... score: (test=0.868) total time= 29.6s
[CV] END ..... score: (test=0.873) total time= 28.5s
[CV] END ..... score: (test=0.873) total time= 30.1s
[CV] END ..... score: (test=0.870) total time= 28.9s
[CV] END ..... score: (test=0.874) total time= 30.6s
AUC-ROC scores for each fold: [0.86754862 0.87348419 0.87335295 0.87040372 0.87358249]
Mean AUC-ROC across all folds: 0.8716743938028738
```

3.3 Estimating AUC using nested CV

Let's try to improve on the results of the random forest by tweaking its hyperparameters. Build a pipeline that computes the nested cv AUC-ROC. Include different values for "n_estimators", "max_samples", "max_features", "max_depth" and "max_leaf_nodes" in your hyperparameter grid. Report the inner and outer fold scores for all combinations of hyperparams and interpret the results. Report the best hyperparams used in each outer fold.

What do you notice about these results? How are these results related to those from the decision tree pipeline? Is hyperparameter optimization worth the trouble?

Hint 1: The pipeline remains the same. You can access the list of all the hyperparams in your pipeline with `pipeline.get_params_keys()`. Notice the naming convention.

Hint 2: `GridSearchCV` and `cross_val_score` are your friends. Specially if you run them with `verbose = 3`.

Hint 3: This gets computationally expensive quickly. Be intelligent about the hyperparam values you include in the grid.

```
In [ ]: ## NESTED CV for Hyperparameter tuning for RF Classifier

# Get total number of features
tot_n_features = len(new_feature_names)

# Define hyperparameter grid
param_grid_rf = {
    'classifier__max_depth': [25],
    'classifier__max_features': ['sqrt'],
    'classifier__max_leaf_nodes': [1000],
    'classifier__n_estimators': [50, 100, 125],
    'classifier__max_samples': [0.5, 0.75, 1.0],
```

```

}

# Initialize inner/outer cross-validation folds
outer_cv_folds = 3
inner_cv_folds = 3

# Initialize lists to store results
outer_scores = []
best_params_list = []

# Perform 5-fold cross-validation
kf = KFold(n_splits=outer_cv_folds, shuffle=True, random_state=0)

# Outer cross-validation Loop
for train_idx, test_idx in kf.split(X_train):
    X_train_fold, X_test_fold = X_train.iloc[train_idx], X_train.iloc[test_idx]
    y_train_fold, y_test_fold = y_train.iloc[train_idx], y_train.iloc[test_idx]

    # Inner cross-validation for hyperparameter tuning
    inner_cv = GridSearchCV(rf_clf, param_grid_rf, cv=inner_cv_folds, scoring='roc_auc')
    inner_cv.fit(X_train_fold, y_train_fold.values.ravel())

    # Best hyperparameters for each outer fold
    best_params = inner_cv.best_params_
    best_params_list.append(best_params)

    # Note: Inner fold CV score for each combination of hyperparameters is printed using
    # Use best model to calculate outer fold score
    best_model = inner_cv.best_estimator_
    y_pred = best_model.predict_proba(X_test_fold)[: , 1]
    outer_score = roc_auc_score(y_test_fold, y_pred)
    outer_scores.append(outer_score)

```

Fitting 3 folds for each of 9 candidates, totalling 27 fits

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=50;; score=0.871 total time= 8.6s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=50;; score=0.876 total time= 8.7s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=50;; score=0.874 total time= 8.5s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=100;; score=0.872 total time= 17.5s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=100;; score=0.877 total time= 17.3s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=100;; score=0.875 total time= 16.6s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.872 total time= 21.1s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.877 total time= 20.8s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.876 total time= 20.7s

```

af_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.875 t
otal time= 21.0s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=50;; score=0.872 t
otal time= 9.8s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=50;; score=0.877 t
otal time= 9.7s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=50;; score=0.874 t
otal time= 9.8s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=100;; score=0.872
total time= 20.2s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=100;; score=0.877
total time= 20.1s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=100;; score=0.875
total time= 20.2s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=125;; score=0.872
total time= 25.0s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=125;; score=0.877
total time= 24.8s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=125;; score=0.875
total time= 24.9s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=50;; score=0.872 to
tal time= 10.9s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=50;; score=0.877 to
tal time= 10.9s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=50;; score=0.875 to
tal time= 10.9s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=100;; score=0.872 t
otal time= 22.2s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=100;; score=0.877 t
otal time= 21.9s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=100;; score=0.875 t
otal time= 22.0s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=125;; score=0.872 t
otal time= 27.1s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=125;; score=0.878 t
otal time= 26.9s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=125;; score=0.875 t
otal time= 27.0s
Fitting 3 folds for each of 9 candidates, totalling 27 fits
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=50;; score=0.873 to
tal time= 8.5s

```

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=50;; score=0.873 total time= 8.4s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=50;; score=0.876 total time= 8.5s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=100;; score=0.874 total time= 16.8s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=100;; score=0.874 total time= 16.8s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=100;; score=0.877 total time= 17.0s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.875 total time= 21.8s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.874 total time= 21.2s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.877 total time= 21.8s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=50;; score=0.874 total time= 10.1s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=50;; score=0.873 total time= 10.2s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=50;; score=0.876 total time= 9.8s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=100;; score=0.875 total time= 19.9s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=100;; score=0.874 total time= 20.2s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=100;; score=0.877 total time= 19.8s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=125;; score=0.875 total time= 24.6s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=125;; score=0.874 total time= 24.7s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=125;; score=0.877 total time= 25.1s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=50;; score=0.875 total time= 11.3s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=50;; score=0.873 total time= 11.5s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=50;; score=0.875 total time= 11.5s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=100;; score=0.875 total time= 22.2s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=100;; score=0.874 total time= 21.6s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=100;; score=0.876 total time= 22.0s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=125;; score=0.875 total time= 27.7s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=125;; score=0.874 total time= 27.5s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=125;; score=0.876 total time= 27.4s

Fitting 3 folds for each of 9 candidates, totalling 27 fits

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=50;; score=0.873 total time= 8.5s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=50;; score=0.876 total time= 8.5s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=50;; score=0.874 total time= 8.5s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=100;; score=0.874 total time= 16.9s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=100;; score=0.877 total time= 17.1s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=100;; score=0.875 total time= 17.1s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.874 total time= 20.9s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.877 total time= 21.2s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.5, classifier__n_estimators=125;; score=0.875 total time= 21.4s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=50;; score=0.873 total time= 9.9s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=50;; score=0.876 total time= 9.9s

[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=50;; score=0.873 total time= 10.2s

[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=100;; score=0.873 total time= 19.8s

[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=100;; score=0.877


```

total time= 20.2s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=100;; score=0.874
total time= 20.1s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=125;; score=0.873
total time= 24.5s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=125;; score=0.877
total time= 25.1s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=0.75, classifier__n_estimators=125;; score=0.874
total time= 25.1s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=50;; score=0.872 to
tal time= 10.9s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=50;; score=0.876 to
tal time= 11.0s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=50;; score=0.875 to
tal time= 10.7s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=100;; score=0.874 t
otal time= 22.1s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=100;; score=0.877 t
otal time= 22.2s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=100;; score=0.875 t
otal time= 21.8s
[CV 1/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=125;; score=0.873 t
otal time= 26.9s
[CV 2/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=125;; score=0.877 t
otal time= 27.0s
[CV 3/3] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_le
af_nodes=1000, classifier__max_samples=1.0, classifier__n_estimators=125;; score=0.875 t
otal time= 27.5s

```

In []:

```

# 1. Report outer fold scores for best model (optimized hyperparameters)
print("Outer fold scores for RF:", outer_scores)
print('Mean outer fold score for RF: ', np.mean(outer_scores))

# 2. Report best hyperparameters used in each outer fold
print("Best hyperparameters for each outer fold for RF:")
for i, params in enumerate(best_params_list):
    print(f"Fold {i+1}: {params}")

```

Outer fold scores for RF: [0.876243407832878, 0.8745580321525952, 0.875708265275702]

Mean outer fold score for RF: 0.8755032350870584

Best hyperparameters for each outer fold for RF:

```

Fold 1: {'classifier__max_depth': 25, 'classifier__max_features': 'sqrt', 'classifier__m
ax_leaf_nodes': 1000, 'classifier__max_samples': 1.0, 'classifier__n_estimators': 100}
Fold 2: {'classifier__max_depth': 25, 'classifier__max_features': 'sqrt', 'classifier__m
ax_leaf_nodes': 1000, 'classifier__max_samples': 0.75, 'classifier__n_estimators': 125}
Fold 3: {'classifier__max_depth': 25, 'classifier__max_features': 'sqrt', 'classifier__m
ax_leaf_nodes': 1000, 'classifier__max_samples': 0.5, 'classifier__n_estimators': 125}

```

1. Report the inner and outer fold scores for all combinations of hyperparams and interpret the results. Report the best hyperparams used in each outer fold.
- The inner fold scores are printed 2 cells above and the outer fold scores are printed in the cell above. The best hyperparameters for each outer fold are reported above. Looking at the spread of scores (standard deviation) in our hyperparameter search, it appears there is little improvement or worsening in model performance with the with the hyperparameter grid we are optimizing over.
1. What do you notice about these results? How are these results related to those from the decision tree pipeline? Is hyperparameter optimization worth the trouble?
- In this hyperparameter optimization, I kept the max_depth around the same size as what was determined optimal from the DT classifier (25 vs 30), used 'sqrt' for max_features to reduce the computation time, and used the optimal max_leaf_nodes (1000) from DT classifier hyperparameter search. I varied the number of estimators and the max number of samples for the RF classifier to determine what the optimal parameters would be for those since these had not been optimized previously. Overall there was little increase in model performance from the DT classifier to the RF classifier (DT mean score = 0.8605 vs RF mean score = 0.8755). Hyperparameter optimization was NOT WORTH THE TROUBLE HERE, I started running 3 difference hyperparameter searches with 3 outer and 3 inner fold nest CV and had to stop them after 1+ hours because it was still not done. There was very little gain in model performance and so such an extensive hyperparameter search was not worth it.

3.4 Take stock of parts II and III

What do you conclude from parts II and III? If you were to choose between a decision tree and a random forest for this classification task, what would be your choice and why?

The Random Forest (RF) classifier technically outperformed the Decision Tree (DT) classifier in terms of predictive accuracy, given the higher AUC-ROC scores obtained in 2.4 vs 3.3. This outperformance, however, was marginal with the difference in mean score being just 0.015. However, on average, the RF classifier achieves better generalization performance and stability due to its ensemble nature, which combines multiple DTs, each with a varying number of features and leaf nodes, thereby reducing the risk of overfitting. However the hyperparameter search was far too computationally expensive and did not justify the gain in model accuracy. For the purposes of this classification task I would use a simpler decision tree model and tune the hyperparameters slightly more to see if I can get a better model performance.

Part IV: Test data, ROC curves and fairness thresholds

Note that until now, we have not used the test data for any purpose. Let's now incorporate it into our analysis and use it to find a "fair" threshold value.

4.1 Find the optimal hyperparameters and fit a decision tree pipeline

Use all of your training data to find the optimal hyperparameters and fit a decision tree pipeline. Report the optimal hyperparameters.

Note: Recall that nested cross-validation is only providing you with an estimate of the out-of-sample performance of the model finding procedure that involves hyperparameter optimization. At training time you should carry out the entire model finding procedure, including hyperparameter optimization.

In []:

```
# Define the hyperparameter grid
tot_n_features = len(new_feature_names)
param_grid_dt = {
    'classifier__max_depth': [15, 25, 50],
    'classifier__max_features': [tot_n_features, 'log2', 'sqrt'],
    'classifier__max_leaf_nodes': [500, 1000, 1500]
}

# Create the grid search object
grid_search = GridSearchCV(estimator=tree_clf, param_grid=param_grid_dt, cv=5, scoring=

# Perform grid search on the entire training data
grid_search.fit(X_train, y_train.values.ravel())

# Get the best hyperparameters
best_params = grid_search.best_params_
print("Optimal hyperparameters:", best_params)
```

```
Fitting 5 folds for each of 27 candidates, totalling 135 fits
[CV 1/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=500;; score=0.862 total time= 3.4s
[CV 2/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=500;; score=0.867 total time= 3.3s
[CV 3/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=500;; score=0.865 total time= 3.3s
[CV 4/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=500;; score=0.863 total time= 3.3s
[CV 5/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=500;; score=0.865 total time= 3.2s
[CV 1/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.857 total time= 4.5s
[CV 2/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.863 total time= 4.4s
[CV 3/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.860 total time= 4.5s
[CV 4/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.859 total time= 4.6s
[CV 5/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=1000;; score=0.861 total time= 4.4s
[CV 1/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.852 total time= 5.1s
[CV 2/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.858 total time= 5.1s
[CV 3/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_lea
f_nodes=1500;; score=0.857 total time= 5.2s
```

[CV 4/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.853 total time= 5.2s

[CV 5/5] END classifier__max_depth=15, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.856 total time= 5.2s

[CV 1/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.738 total time= 0.2s

[CV 2/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.802 total time= 0.2s

[CV 3/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.729 total time= 0.2s

[CV 4/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.762 total time= 0.2s

[CV 5/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.734 total time= 0.2s

[CV 1/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.738 total time= 0.2s

[CV 2/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.802 total time= 0.3s

[CV 3/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.729 total time= 0.3s

[CV 4/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.762 total time= 0.3s

[CV 5/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.734 total time= 0.3s

[CV 1/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.738 total time= 0.3s

[CV 2/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.802 total time= 0.3s

[CV 3/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.729 total time= 0.3s

[CV 4/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.762 total time= 0.3s

[CV 5/5] END classifier__max_depth=15, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.734 total time= 0.2s

[CV 1/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.812 total time= 0.3s

[CV 2/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.802 total time= 0.3s

[CV 3/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.826 total time= 0.3s

[CV 4/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.819 total time= 0.3s

[CV 5/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.806 total time= 0.3s

[CV 1/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.812 total time= 0.4s

[CV 2/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.802 total time= 0.4s

[CV 3/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.827 total time= 0.4s

[CV 4/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.819 total time= 0.4s

[CV 5/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.805 total time= 0.3s

[CV 1/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.812 total time= 0.4s

[CV 2/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.802 total time= 0.4s

[CV 3/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.826 total time= 0.4s

[CV 4/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.818 total time= 0.5s

[CV 5/5] END classifier__max_depth=15, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.806 total time= 0.4s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.862 total time= 3.7s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.869 total time= 3.8s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.867 total time= 3.9s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.866 total time= 3.7s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.868 total time= 3.7s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.859 total time= 5.5s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.865 total time= 5.5s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.863 total time= 5.5s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.862 total time= 5.2s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.863 total time= 5.4s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.854 total time= 6.7s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.860 total time= 6.6s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.857 total time= 6.7s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.857 total time= 6.5s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.858 total time= 6.6s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.790 total time= 0.3s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.819 total time= 0.3s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.770 total time= 0.3s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.787 total time= 0.3s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.777 total time= 0.3s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.789 total time= 0.4s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.820 total time= 0.4s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.770 total time= 0.3s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.792 total time= 0.3s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.779 total time= 0.3s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.789 total time= 0.4s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.819 total time= 0.4s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.769 total time= 0.4s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.792 total time= 0.4s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.799 total time= 0.4s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.844 total time= 0.4s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.812 total time= 0.4s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.840 total time= 0.4s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.838 total time= 0.4s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.834 total time= 0.4s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.844 total time= 0.5s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.818 total time= 0.5s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.839 total time= 0.5s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.837 total time= 0.5s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.834 total time= 0.5s

[CV 1/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.844 total time= 0.6s

[CV 2/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.821 total time= 0.6s

[CV 3/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.839 total time= 0.6s

[CV 4/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.841 total time= 0.6s

[CV 5/5] END classifier__max_depth=25, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.833 total time= 0.5s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.862 total time= 3.8s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.869 total time= 3.9s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.867 total time= 3.9s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.866 total time= 3.9s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=500;; score=0.868 total time= 3.7s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.859 total time= 5.4s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.865 total time= 5.5s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.863 total time= 5.6s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.863 total time= 5.5s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1000;; score=0.863 total time= 5.6s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.854 total time= 6.8s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.860 total time= 6.8s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.857 total time= 6.9s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.858 total time= 6.9s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=287, classifier__max_leaf_nodes=1500;; score=0.858 total time= 6.9s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.799 total time= 0.3s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.825 total time= 0.3s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.789 total time= 0.3s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.802 total time= 0.3s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=500;; score=0.788 total time= 0.3s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.804 total time= 0.4s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.831 total time= 0.4s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.804 total time= 0.4s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.809 total time= 0.4s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1000;; score=0.789 total time= 0.4s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.803 total time= 0.5s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.829 total time= 0.5s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.806 total time= 0.5s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.811 total time= 0.5s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=log2, classifier__max_leaf_nodes=1500;; score=0.800 total time= 0.5s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.845 total time= 0.4s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.819 total time= 0.4s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.840 total time= 0.4s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.836 total time= 0.4s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=500;; score=0.847 total time= 0.4s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.847 total time= 0.5s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.823 total time= 0.5s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.844 total time= 0.6s

[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.838 total time= 0.5s

[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1000;; score=0.850 total time= 0.6s

[CV 1/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.846 total time= 0.7s

[CV 2/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.827 total time= 0.6s

[CV 3/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_leaf_nodes=1500;; score=0.843 total time= 0.7s

```
[CV 4/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.839 total time= 0.7s
[CV 5/5] END classifier__max_depth=50, classifier__max_features=sqrt, classifier__max_le
af_nodes=1500;; score=0.850 total time= 0.7s
Optimal hyperparameters: {'classifier__max_depth': 50, 'classifier__max_features': 287,
'classifier__max_leaf_nodes': 500}
```

The Optimal hyperparameters for this DT classifier are : {'classifier__max_depth': 50,
'classifier__max_features': 287, 'classifier__max_leaf_nodes': 500}

4.2 Scores and the ROC curve

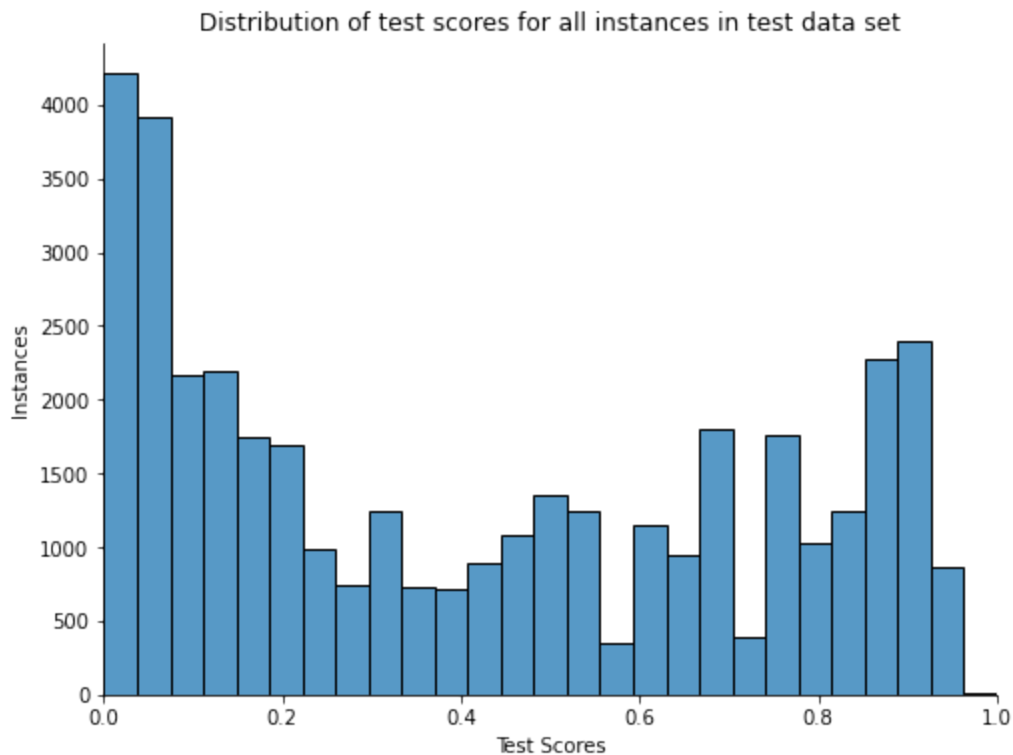
Use the fitted pipeline to compute score values for all instances in the test set. Show the distribution of the scores in a histogram. Additionally, compute the (FPR,TPR) combinations for all relevant threshold values and use them to plot the ROC curve in a different figure. Following the convention, include a dashed line along the diagonal. Remember to label the axes and to make the figures as polished as possible.

Note 1: You should NOT be using `sklearn.metrics.auc` or `sklearn.metrics.RocCurveDisplay` to calculate or display the ROC curve. Please code this part yourself.

```
In [ ]: # Compute score values (y_hat predict) for all instances in the test set
best_model = grid_search.best_estimator_
test_scores = best_model.predict_proba(X_test)[:, 1]

# Plot the distribution of scores in a histogram
plt.figure(figsize=(8, 6))
sns.histplot(test_scores)
plt.xlabel('Test Scores')
plt.xlim([0,1])
sns.despine()
plt.ylabel('Instances')
plt.title('Distribution of test scores for all instances in test data set')
```

```
Out[ ]: Text(0.5, 1.0, 'Distribution of test scores for all instances in test data set')
```

In []:

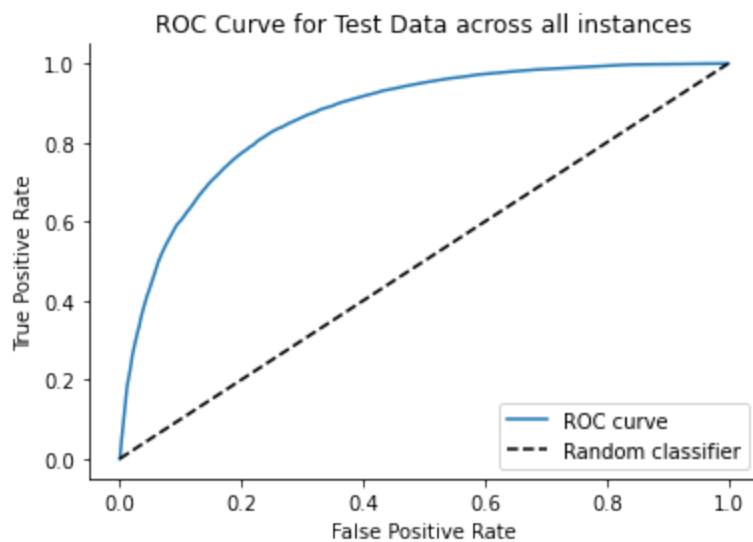
```
# Compute FPR vs TPR combinations for all threshold values
from sklearn.metrics import confusion_matrix

# Define threshold array from 0 to 1
thresholds = np.linspace(0, 1, 100)

# Get predicted outcome based on given threshold array
y_pred = (test_scores[:, np.newaxis] >= thresholds).astype(int)

# Calculate the tpr and fpr across all threshold values
fpr_all = []
tpr_all = []
for idx, threshold in enumerate(thresholds):
    tn, fp, fn, tp = confusion_matrix(y_test, y_pred[:,idx]).ravel()
    tpr = tp / (tp + fn)
    fpr = fp / (fp + tn)
    fpr_all.append(fpr)
    tpr_all.append(tpr)

# Plot the ROC curve
sns.lineplot(x = fpr_all, y = tpr_all, label='ROC curve')
plt.plot([0, 1],[0, 1], color='black', linestyle='--', label='Random classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for Test Data across all instances')
plt.legend()
sns.despine()
```



4.3 ROC curves by racial groups

Compute and plot in the same figure the ROC curves for the following racial groups identified by the ACS:

- White (RAC1P==1) (plot in blue)
- African American (RAC1P==2) (plot in orange)
- Asian American (RAC1P==6) (plot in green)

Interpret the results.

```
In [ ]: def ROC_curve(rf_clf, X_test, y_test, indices):
    """
    Function for computing TPR and FPR for x_test and y_test given set of indices speci
    """
    # Define sub population to analyze based on provided indices
    X_test_new = X_test.loc[indices]
    y_test_new = y_test.loc[indices]

    # calculate test scores on sub population
    test_scores = rf_clf.predict_proba(X_test_new)[: , 1]

    # Define threshold array from 0 to 1
    thresholds = np.linspace(0, 1, 100)

    # Get predicted outcome based on given threshold array
    y_pred = (test_scores[: , np.newaxis] >= thresholds).astype(int)

    # Calculate the tpr and fpr across all threshold values
    fpr_all = []
    tpr_all = []
    for idx, threshold in enumerate(thresholds):
        tn, fp, fn, tp = confusion_matrix(y_test_new, y_pred[:,idx]).ravel()
        tpr = tp / (tp + fn)
        fpr = fp / (fp + tn)
        fpr_all.append(fpr)
        tpr_all.append(tpr)
    return fpr_all, tpr_all
```

In []:

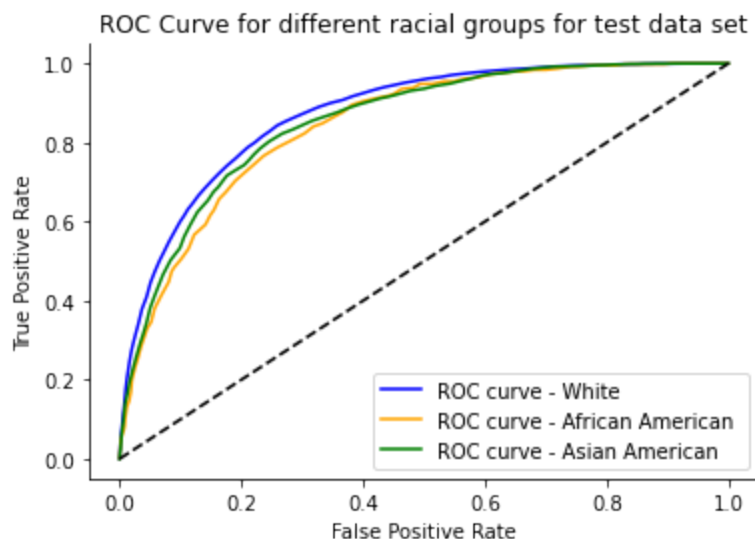
```

# Specify subset of X_test indices for given racial population
idxs_white = X_test[X_test['RAC1P'] == 1].index.values
idxs_afr = X_test[X_test['RAC1P'] == 2].index.values
idxs_asian = X_test[X_test['RAC1P'] == 6].index.values

# Compute fpr and tpr for each racial group
fpr_white, tpr_white = ROC_curve(rf_clf, X_test, y_test, idxs_white)
fpr_afr, tpr_afr = ROC_curve(rf_clf, X_test, y_test, idxs_afr)
fpr_asian, tpr_asian = ROC_curve(rf_clf, X_test, y_test, idxs_asian)

# Plot the ROC curves for each racial group
sns.lineplot(x = fpr_white, y = tpr_white, color = 'blue', label='ROC curve - White')
sns.lineplot(x = fpr_afr, y = tpr_afr, color = 'orange', label='ROC curve - African Amer
sns.lineplot(x = fpr_asian, y = tpr_asian, color = 'green', label='ROC curve - Asian Ame
plt.plot([0, 1],[0, 1], color='black', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for different racial groups for test data set')
plt.legend()
sns.despine()

```



Results interpretation for 4.3

- Analyzing the figure above we first notice that the AUC for the african-american curve is the smallest, followed by the asian-american curve, while the white ROC curve has the largest area. Since the AUC of the ROC curve is an indication of model performance in predicting the target, we would say that the model performs better at predicting the target (whether an individual's income is above \$50,000) better for the white population vs the african american and asian american population. To further check if the model violates equality of opportunity we could compute this metric as the white_TPR - african-american_TPR or white_TPR - asian-american_TPR.

4.4 Achieving error parity

Implement a thresholding strategy that satisfies error parity for all racial groups with $FPR = 0.3$, $TPR = 0.6$ and $\epsilon = 0.025$. In plain english, find a way of setting thresholds for the members of each group in the test data that, when evaluated on the test data, delivers FPR and TPR values that differ at most from the objective values by ϵ . Plot the estimated TPRs and FPRs of the racial groups in the ROC plot. Use star markers, colored accordingly.

Hint: Consider using group-specific stochastic thresholds.

Old Solution attempt

```
In [ ]: def find_threshold(fpr, tpr, thresholds, desired_fpr, desired_tpr, epsilon):
    """
    Function: determines optimal threshold based of desired fpr and desired tpr so they
    """
    # convert lists to arrays
    fpr = np.array(fpr)
    tpr = np.array(tpr)

    # Find the indices of relevant thresholds
    relevant_indices = np.where((abs(fpr - desired_fpr) <= epsilon) & (abs(tpr - desired_tpr) <= epsilon))

    # Initialize variables to store the optimal threshold and the minimum difference
    optimal_threshold = None
    min_difference = float('inf') # set as arbitrarily large

    # Iterate over the relevant indices
    for i in relevant_indices:
        # Calculate the total difference: sum of the FPR and TPR differences
        total_difference = abs(fpr[i] - desired_fpr) + abs(tpr[i] - desired_tpr)

        # Check if the total difference is smaller than the current minimum difference
        if total_difference < min_difference:
            # Update the optimal threshold and minimum difference
            optimal_threshold = thresholds[i]
            min_difference = total_difference
            idx = i

    return optimal_threshold, idx

# Define threshold values used above
thresholds = np.linspace(0, 1, 100)

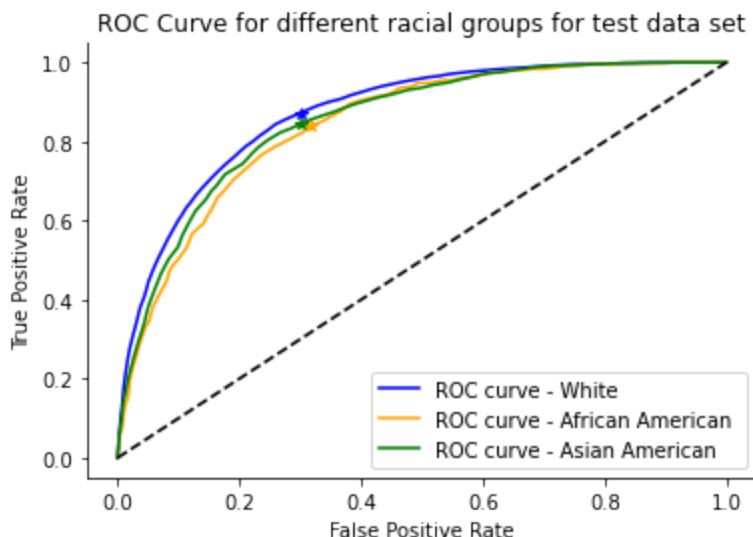
# Define the desired FPR, TPR, and epsilon values
desired_fpr = 0.3
desired_tpr = 0.85 # NOTE: I had to change this so it was within the AUC curves for the
epsilon = 0.025

# Find the threshold that satisfies error parity
optimal_threshold_white, idx_white = find_threshold(fpr_white, tpr_white, thresholds, desired_fpr, desired_tpr, epsilon)
optimal_threshold_afr, idx_afr = find_threshold(fpr_afr, tpr_afr, thresholds, desired_fpr, desired_tpr, epsilon)
optimal_threshold_asia, idx_asian = find_threshold(fpr_asian, tpr_asian, thresholds, desired_fpr, desired_tpr, epsilon)
```

```
In [ ]: # Plot the ROC curves for each racial group
sns.lineplot(x = fpr_white, y = tpr_white, color = 'blue', label='ROC curve - White')
plt.plot(fpr_white[idx_white], tpr_white[idx_white], color = 'blue', marker = '*')
sns.lineplot(x = fpr_afr, y = tpr_afr, color = 'orange', label='ROC curve - African American')
```

```
plt.plot(fpr_afr[idx_afr],tpr_afr[idx_afr], color = 'orange', marker = '*')
sns.lineplot(x = fpr_asian, y = tpr_asian,color = 'green', label='ROC curve - Asian Ame
plt.plot(fpr_asian[idx_asian],tpr_asian[idx_asian], color = 'green', marker = '*')

plt.plot([0, 1],[0, 1], color='black', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for different racial groups for test data set')
plt.legend()
sns.despine()
```



4.5 Improving the results [extra-credit]

Can you improve the results from 4.4? That is, can you tweak your algorithm to deliver a higher TPR and lower FPR while still satisfying error parity with $\epsilon = 0.025$? What is the best result that you are able to achieve? Is there a hard limit on how much you can improve?

In []:

```
# your code here
```

your observations here

4.6 Conclusion

What can you say about welfare and error parity? Some things to consider:

- In this income prediction task, is enforcing error parity costly?
- Would you expect this results to generalize to other predictions problems?
- Is the group definition relevant?
- Who benefits from enforcing error parity? Who doesn't?

1. What can you say about welfare and error parity? Some things to consider:

- Error parity aims to promote fairness by ensuring that prediction errors are balanced across different demographic groups, like racial groups in this prediciton problem. By mitigating

disparities in model predictions and considering different groups, error parity ensures the welfare of protected/disadvantaged groups.

1. In this income prediction task, is enforcing error parity costly?

- Enforcing error parity is costly in the sense that it requires careful consideration fairness constraints. This can lead to more complex model development and an evaluation processes that may impose negative constraints on model performance. Hence, leading to a trade-of between predictive accuracy and performance.

1. Would you expect this results to generalize to other predictions problems?

- No I would not expect that the absolute results from this problem would generalize to other prediction problems with the ACS data set or beyond since things will vary depending on the prediction task. Tghe specific context and data set need to be appropriately considered. However the methodology used to achieve error parity can be applied to other prediction problems to achieve equitable distribution of outcomes.

1. Is the group definition relevant?

- The group definition for this prediction problem is relevant since we predicting an individual's income above a certain thershold. In general, group definition should be selected based on the specific context and goals of the prediction task and should take into account any fairness objectives set. In our case, race is a meaningful group definition in order to identify and mitigate biases in this income prediction model.

1. Who benefits from enforcing error parity? Who doesn't?

- enforcing error parity can benefit marginalized groups by promoting fairness and reducing discrimination. This is crucial for adressing long standing biases in decision-making processes that have to do with generating policies that impact the population. The case where error parity may not be beneficial is if it negatively impacts model performance and accuracy, which could lead to disadvantaging certain groups because of decreased predictive power. groups that currrently benefit from existing biases in models could see error parity as disavantageous.